

torch.nn.functional.upsample#

<https://pytorch.org/docs/stable/generated/torch.nn.functional.upsample.html>

Description:

Provided tensor is upsampled to either the given size or the given scale_factor. This function is deprecated in favor of `torch.nn.functional.interpolate()`. This is equivalent with `nn.functional.interpolate(...)`.

Function Signature

```
torch.nn.functional.upsample(input, size=None,
scale_factor=None, mode='nearest', align_corners=None)
[source]#
```

Parameters

Notes & Warnings

WARNING

This function is deprecated in favor of `torch.nn.functional.interpolate()`. This is equivalent with `nn.functional.interpolate(...)`.

NOTE

This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information.

NOTE

With mode='bicubic', it's possible to cause overshoot, in other words it can produce negative values or values greater than 255 for images. Explicitly call `result.clamp(min=0, max=255)` if you want to reduce the overshoot when displaying the image.

WARNING

With `align_corners = True`, the linearly interpolating modes (linear, bilinear, and trilinear) don't proportionally align the output and input pixels, and thus the output values can depend on the input size. This was the default behavior for these modes up to version 0.3.1. Since then, the default behavior is `align_corners = False`. See Upsample for concrete examples on how this affects the outputs.

Detailed Documentation

Documentation

Provided tensor is upsampled to either the given size or the given `scale_factor`

This function is deprecated in favor of `torch.nn.functional.interpolate()`. This is equivalent with `nn.functional.interpolate(...)`.

This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information.

The algorithm used for upsampling is determined by `mode`.

Currently temporal, spatial and volumetric upsampling are supported, i.e. expected inputs are 3-D, 4-D or 5-D in shape.

The input dimensions are interpreted in the form: mini-batch x channels x [optional depth] x [optional height] x width.

The modes available for upsampling are: nearest, linear (3D-only), bilinear, bicubic (4D-only), trilinear (5D-only)

`input` (Tensor) – the input tensor

`size` (int or Tuple[int] or Tuple[int, int] or Tuple[int, int, int]) – output spatial size.

`scale_factor` (float or Tuple[float]) – multiplier for spatial size. Has to match input size if it is a tuple.

`mode` (str) – algorithm used for upsampling: 'nearest' | 'linear' | 'bilinear' | 'bicubic' | 'trilinear'. Default: 'nearest'

`align_corners` (bool, optional) – Geometrically, we consider the pixels of the input and output as squares rather than points. If set to True, the input and output tensors are

aligned by the center points of their corner pixels, preserving the values at the corner pixels. If set to `False`, the input and output tensors are aligned by the corner points of their corner pixels, and the interpolation uses edge value padding for out-of-boundary values, making this operation independent of input size when `scale_factor` is kept the same. This only has an effect when `mode` is `'linear'`, `'bilinear'`, `'bicubic'` or `'trilinear'`.
Default: `False`

With `mode='bicubic'`, it's possible to cause overshoot, in other words it can produce negative values or values greater than 255 for images. Explicitly call `result.clamp(min=0, max=255)` if you want to reduce the overshoot when displaying the image.

With `align_corners = True`, the linearly interpolating modes (`linear`, `bilinear`, and `trilinear`) don't proportionally align the output and input pixels, and thus the output values can depend on the input size. This was the default behavior for these modes up to version 0.3.1. Since then, the default behavior is `align_corners = False`. See [Upsample](#) for concrete examples on how this affects the outputs.